

Sistemas Operativos: Práctica 2

Shaofan Xu

Javier Maria Santa

9 de abril de 2026

1. Introducción

El objetivo de esta práctica es utilizar mecanismos de sincronización para simular una block chain. En este entorno diversos procesos se forman mineros para competir por el premio, así utilizando señales y semáforos para comunicar entre los procesos.

2. Requisitos cumplidos y no cumplidos

2.1. Requisitos cumplidos

Se han implementado correctamente los siguientes objetivos principales de la práctica:

- **Gestión de múltiples procesos Minero:** Se ha implementado un fichero compartido protegido por mutex donde los procesos registran pid al entrar al sistema y eliminan al salir del sistema cuando llegan su tiempo límite, así destruyendo el fichero el último proceso que sale.
- **Proceso de minería:** el primer proceso establece un objetivo inicial (0 en nuestro caso), y lanza señal para iniciar la ronda, posteriormente el primer minero que se encuentra el resultado escribe su resultado en `target.tgt` y lanzando otra señal para empezar la votación.
- **Sistema de votación:** una vez empezado el proceso de votación los procesos participantes deben comprobar el resultado del minero candidato para votar si esta a favor o en contra, en cuyo caso, el proceso candidato irá comprobando los votos teniendo un máximo número de intento hasta tener más de un 50 % de voto a favor. Una vez tenido el resultado, el proceso candidato convierte en ganador, guardando su resultado en un fichero cuyo nombre es su pid, y el target de la siguiente ronda en `target.tgt`
- **Protección de zona crítica:** Para que el sistema no falle por problemas de concurrencia ni pierda señales por el camino, hemos protegido las zonas críticas del código de cuatro formas:
 - **Bloqueo inicial de señales:** Antes de entrar al bucle principal, se crea una máscara con `SIGUSR1`, `SIGUSR2` y `SIGALRM` y usando `sigprocmask`. Así nos aseguramos de que, si llega una señal justo cuando esta leyendo o escribiendo en un fichero compartido, esta se quede pendiente y no rompe la ejecución a medias.
 - **Control de interrupciones (EINTR):** Al mezclar semáforos y señales, pasaba que a veces la llamada a `sem_wait()` se interrumpía por la llegada de una señal antes de haber conseguido el semáforo. Para evitar que el código siga adelante asumiendo que tiene el recurso (lo que corrompería todo), protegemos las peticiones en un bucle: `while (sem_wait(...) == -1 && errno == EINTR);`. Así, si una señal despierta al proceso por error, este vuelve a intentar.

- **Espera pasiva con `sigsuspend`:** Para cumplir con el enunciado y no freír la CPU con esperas activas, cuando el minero no está minando, ni votando, ni gestionando la victoria, se pone a dormir llamando a `sigsuspend(&mask_empty)`. Esto vacía la máscara de bloqueos temporalmente y deja al proceso suspendido hasta que llega una señal que lo despierta para la siguiente fase.
- **Evitar deadlocks en los hilos:** Durante las pruebas a menudo sucede que un proceso estaba bloqueado en el `pthread_join` esperando a sus hilos y otro minero ganaba la ronda, el proceso bloqueado no podía recibir la señal `SIGUSR2` para ir a votar (se quedaba colgado). Como solución propuesta se desbloquea las señales con `SIG_UNBLOCK` justo antes del join y volviéndolas a bloquear después. De esta forma, la orden de votación entra sin problemas, los hilos abortan su ejecución y el programa no se atasca.

2.2. Requisitos no cumplidos

Todos los requisitos obligatorios especificados en el enunciado han sido implementados y funcionan correctamente.

3. Pruebas: Funcionamiento y Rendimiento

3.1. Entorno de ejecución

Las pruebas y mediciones se han llevado a cabo bajo las siguientes condiciones de hardware y software para garantizar la consistencia de los resultados:

- Sistema Operativo: WSL2 (Ubuntu) sobre Windows 11.
- Hardware: 16 GB de memoria RAM.
- Herramienta de medición: Comando `time` de la shell de Linux.

3.2. Pruebas de Funcionamiento

Se han ejecutado diversas pruebas para asegurar el correcto funcionamiento del programa.

3.2.1. Casos normales

Se ejecutó el programa con parámetros correspondientes:

```
./miner 3 8 & ./miner 5 8 & ./miner 10 8 & ./miner 10 1
```

El programa generó correctamente los archivo `.log` con la estructura especificada (Id, ronda, target, solución, etc), y la validación (accepted/rejected). Así imprimiendo en la terminal exactamente los mensajes requeridos:

```
● → practica2 git:(main) x make test
./miner 3 8 & ./miner 5 8 & ./miner 10 8 & ./miner 10 1
Miner 48017 added to system
Miner 48014 added to system
Miner 48016 added to system
Miner 48015 added to system
Winner 48016 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48016 -> [ Y Y Y ] -> Accepted
Winner 48016 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48016 -> [ Y Y Y ] -> Accepted
Winner 48016 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Miner 48014 exited system. Remaining miners: 3
Miner 48015 exited system. Remaining miners: 2
Miner 48017 exited system. Remaining miners: 1
Miner 48016 was the last one. System cleaned.
```

Figura 1: Ejecución de programa demostrando el proceso de entrada de los procesos mineros, los procesos minando, y destrucción del fichero cuando termina el último proceso


```
dsadas@Santa:/mnt/c/Users/jsant/OneDrive/Documentos/Prog/Tercero/SOPER/I2/S0/practica2$ tail -n 15 8129.txt

Id:      39
Winner:  8129
Target:  3419216
Solution: 3308492 (validated)
Votes:   3/3
Wallets: 8129:39

Id:      40
Winner:  8129
Target:  4505593
Solution: 8924837 (validated)
Votes:   3/3
Wallets: 8129:40

dsadas@Santa:/mnt/c/Users/jsant/OneDrive/Documentos/Prog/Tercero/SOPER/I2/S0/practica2$
```

Figura 3: Registro de victorias generado por el proceso 8129. Como se puede observar, el minero ha registrado correctamente el 'Target' y la 'Solution' de cada ronda, contabilizando los votos a favor (Y) de los demás procesos. El campo 'Wallets' demuestra que el incremento de monedas locales se gestiona de forma segura tras la validación de la red.

```

→ practica2 git:(main) x ./miner 8 4 & sleep 3 ; ./miner 2 4
[1] 8701
Miner 8701 added to system
Miner 8709 added to system
Winner 8701 => [ Y ] => Accepted
Winner 8701 => [ Y ] => Accepted
Winner 8701 => [ Y ] => Accepted
Winner 8701 => [ Y ] => Accepted
Winner 8701 => [ Y ] => Accepted
Winner 8701 => [ Y ] => Accepted
Winner 8701 => [ Y ] => Accepted
Winner 8701 => [ Y ] => Accepted
Winner 8701 => [ Y ] => Accepted
Winner 8701 => [ Y ] => Accepted
Winner 8701 => [ Y ] => Accepted
Winner 8701 => [ Y ] => Accepted
Winner 8701 => [ Y ] => Accepted
Winner 8701 => [ Y ] => Accepted
Winner 8709 => [ Y ] => Accepted
Miner 8709 exited system. Remaining miners: 1
→ practica2 git:(main) x Winner 8701 => [ ] => Accepted
Miner 8701 was the last one. System cleaned.

[1] + 8701 done      ./miner 8 4

```

Figura 4: La ejecución demuestra que el sistema es totalmente robusto y no sufre *deadlocks*: Ambos procesos se registran correctamente y logran votar de forma concurrente ([Y] ->Accepted). Cuando el minero de 2 segundos agota su alarma, sale de forma ordenada, actualiza el contador de procesos restantes y libera sus recursos sin interrumpir al otro. El minero de 8 segundos detecta inmediatamente que se ha quedado solo. Cumpliendo con los requisitos, el sistema entra en espera pasiva sin consumir recursos de forma innecesaria hasta que se agota su tiempo y limpia el sistema. Al expirar el tiempo del último minero, este detecta que es el único que queda en el sistema y se encarga de ejecutar los `sem_unlink` y eliminar los ficheros `.tgt`, `.pid` y `.txt`, dejando el entorno completamente limpio (System cleaned) y evitando fugas de memoria en el sistema operativo.

3.4. Pruebas con valgrind

Para garantizar la robustez del sistema, se analizó el código utilizando Valgrind (se puede ejecutar con `make valgrind`). Los resultados confirmaron que no existen fugas de memoria tras la limpieza del último minero.

```

• parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/S0/practica2$ make valgrind
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes --log-file="val_log_%p.txt" ./miner 5 4 & \
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes --log-file="val_log_%p.txt" ./miner 8 2 & \
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes --log-file="val_log_%p.txt" ./miner 10 2
Miner 12531 added to system
Miner 12530 added to system
Miner 12532 added to system
Winner 12532 => [ Y Y ] => Accepted
Winner 12532 => [ Y Y ] => Accepted
Winner 12531 => [ Y Y ] => Accepted
Winner 12531 => [ Y Y ] => Accepted
Miner 12530 exited system. Remaining miners: 2
Winner 12531 => [ Y ] => Accepted
Winner 12532 => [ Y ] => Accepted
Winner 12531 => [ Y ] => Accepted
Miner 12531 exited system. Remaining miners: 1
Winner 12532 => [ ] => Accepted
Miner 12532 was the last one. System cleaned.

```

Figura 5: En la captura, muestra la ejecución del programa con valgrind, donde las salidas de valgrind se ha redireccionado a ficheros

```

practica2 > ≡ val_log_12530.txt
1  ==12530== Memcheck, a memory error detector
2  ==12530== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
3  ==12530== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
4  ==12530== Command: ./miner 5 4
5  ==12530== Parent PID: 12529
6  ==12530==
7  ==12530==
8  ==12530== HEAP SUMMARY:
9  ==12530==      in use at exit: 0 bytes in 0 blocks
10 ==12530==    total heap usage: 57 allocs, 57 frees, 80,523 bytes allocated
11 ==12530==
12 ==12530== All heap blocks were freed -- no leaks are possible
13 ==12530==
14 ==12530== For lists of detected and suppressed errors, rerun with: -s
15 ==12530== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
16

```

Figura 6: Ejemplo de salida de valgrind, donde podemos ver que no hay problema de fuga de memoria

4. Conclusión y Respuesta a las cuestiones del enunciado

Pregunta a: ¿Se produce algún problema de concurrencia en el sistema descrito?

Sí, el sistema tiene varios puntos donde la concurrencia puede causar fallos. El problema principal es el acceso a los ficheros compartidos como `pids.pid`, `target.tgt` y `vote.txt`, ya que al ser procesos independientes, varios podrían intentar escribir

o leer a la vez, corrompiendo los datos. También hay un problema de “condición de carrera” al final de cada ronda: varios mineros pueden encontrar una solución válida casi al mismo tiempo y ambos intentarían proclamarse ganadores simultáneamente.

Pregunta b: ¿Cómo se ha solucionado en el planteamiento dado?

La solución se basa en el uso de semáforos con nombre de POSIX que actúan como mutex para garantizar la exclusión mutua. Cada vez que un proceso necesita acceder a un fichero, debe adquirir el semáforo correspondiente (`sem_pid`, `sem_target`, `sem_votes`). Para la competición por la victoria, se utiliza el semáforo `sem_winner` con la función `sem_trywait`, lo que asegura que solo el primer minero en realizar la llamada se convierta en el ganador oficial. Asimismo, se implementó un bloqueo de señales y el uso de `sigsuspend` para evitar la pérdida de notificaciones críticas durante la ejecución de estas secciones.

Pregunta c: ¿Es sencillo, o siquiera factible, organizar un sistema de este tipo usando únicamente señales?

No es nada sencillo y, en la práctica, no es factible para un sistema así. Las señales son solo avisos; no pueden enviar datos como la solución del hash o los votos de cada minero. Además, las señales no sirven para bloquear el acceso a un archivo; si usáramos solo señales, no tendríamos forma de garantizar que dos procesos no escriban en el mismo fichero a la vez, lo que acabaría rompiendo toda la lógica de la cadena.

Pregunta d: ¿Cuáles son los mineros que tienden a ganar más monedas?

Como se puede ver en las pruebas de rendimiento (ver la Figura 2), los mineros que más ganan son los que tienen más hilos (`N_THREADS`). Al tener más hilos, el proceso divide el trabajo de búsqueda y puede probar muchos más hashes por segundo en paralelo. En la captura de la Figura 2, se ve claramente que los procesos 8128 y 8129 (lanzados con 8 hilos) ganan casi todas las rondas, mientras que los de 1 hilo (8130 y 8131) casi nunca llegan a tiempo de encontrar la solución.